

周柄名答辩闭环：语法分析与解析状态

1. 先记住自己的定位

你这部分的核心不是“把整个文法从头背到尾”，而是守住三件事：

1. Bison 怎么把 token 归约成 AST。
2. 语法分析能拦哪些错误，输出是什么。
3. `ParserState` 和 panic-mode 恢复在 parser 里各自负责什么。

最短自我介绍可以直接说：

我主要参与的是语法分析与解析状态这一层。词法分析输出 token 之后，Bison 按 Pascal-S 文法做归约，并在归约动作里构造 AST。语法分析阶段的输出不是目标代码，而是一棵统一的抽象语法树，同时也会在缺分号、结构不完整等情况下给出语法错误并进行有限的 panic-mode 恢复。

2. 现场先打开哪些文件

建议只记这 3 个文件，够用了：

1. [src/parser.y](#)
2. [src/parser_state.h](#)
3. [src/parser_state.cpp](#)

打开顺序建议：

1. 先看 `prog`、`compound_stmt`、`stmt` 这些核心规则，讲“输出 AST”。
2. 再看 `ParserState`，讲“解析期临时状态放在哪里”。
3. 最后看 `finish_syntax_recovery()`、`yyerror()` 和 `error` 规则，讲“有限的 panic-mode 恢复怎么做”。

3. 老师最可能问什么

问题 A：语法分析是怎么实现的？

先打开：

- [src/parser.y:183](#)
- [src/parser.y:185](#)

回答模板：

语法分析用的是 Bison。

词法分析器 `yylex()` 提供 token，`yyparse()` 按文法规则归约。

在每条重要规则的归约动作里，我们直接构造对应的 AST 节点，比如程序根节点、赋值语句、条件语句、函数调用等。

所以语法分析阶段的结果不是文本，而是统一的 AST。

如果老师追问“整个程序的根在哪里构造”：

在 `prog` 规则里构造 `ProgramNode`，把前面暂存的声明和主程序体挂进去。

问题 B：语法分析输出的是什么？

先打开：

- [src/parser.y:185](#)

回答模板：

输出是 AST，也就是抽象语法树。

在 `prog` 规则完成时，`root_ast` 会被设成 `ProgramNode`，里面再包含声明列表和主程序体。后续语义分析和代码生成都从这棵树开始工作。

问题 C：你们能检测哪些语法错误？

先打开：

- [src/parser.y:245](#)
- [src/parser.y:280](#)
- [src/parser.y:351](#)
- [src/parser.y:472](#)
- [src/parser.y:487](#)

回答模板：

能检测的语法错误主要包括：缺失分号、声明项结构不完整、`begin/end` 或 `repeat/until` 结构不完整、语句序列中间断裂、表达式或参数列表不完整等。

这些错误一部分是 Bison 默认发现的，一部分是我们在 `error` 产生式里显式设置同步点后捕获并恢复的。

这里最好补一句：

语法阶段只关心结构合法性，不做类型合法性判断，那是语义分析阶段负责的。

问题 D：`ParserState` 是干什么的？

先打开：

- [src/parser_state.h:9](#)
- [src/parser_state.cpp:11](#)

回答模板：

`ParserState` 是解析期间的上下文状态。

因为有些文法规则在归约时需要暂存参数列表、局部变量声明、数组信息、`record` 字段信息、当前语句列表等，所以我们把这些侧带状态集中放在 `ParserState` 里，而不是散落成很多裸全局变量。

每次开始新一轮解析时，会先 `reset()`，保证状态干净。

如果老师追问“它是不是语义分析器”：

不是。它只服务于 `parser` 的建树过程，不做符号表插入和类型检查。

问题 E：为什么说 parser 只建树？

先打开：

- [src/parser.y:21](#)

回答模板：

现在 `parser.y` 的职责是按文法构造 AST，不再做符号表副作用和类型判定。

这样语法结构和语义检查就分开了，后续扩展 `record`、`repeat-until`、错误恢复时更容易维护。

这句是老师很爱听的“架构意识”。

问题 F：panic-mode 恢复是怎么做的？

先打开：

- [src/parser.y:47](#)
- [src/parser.y:472](#)
- [src/parser.y:482](#)
- [src/parser.y:730](#)

标准答法：

我们在 `parser` 里实现的是有限的 panic-mode 恢复。

具体做法是用 Bison 的 `error` 产生式，把分号、`END`、`UNTIL` 等位置作为同步点。发生语法错误后，先进入 panic-mode，清理解析期侧带状态，再在同步点重新继续后续独立语法片段的分析。

`yyerror()` 负责记录当前语法错误并进入 panic-mode，真正的恢复和继续分析发生在这些 `error` 规则上。

这题最容易答错的地方是：

不要说恢复是在 `error.cpp` 做的。

正确说法：`parser` 负责恢复，`ErrorHandler` 只负责错误输出和抑制重复报错。

4. 最该会指给老师看的 6 段代码

4.1 `prog`：程序根节点

位置：

- [src/parser.y:185](#)

怎么讲：

这条规则对应整个 Pascal-S 程序。这里创建 `ProgramNode`，再把前面暂存的函数声明、变量声明和主程序体统一挂上去。

4.2 compound_stmt 和 stmt_seq

位置:

- [src/parser.y:461](#)
- [src/parser.y:480](#)

怎么讲:

`compound_stmt` 对应 `begin ... end`, `stmt_seq` 对应语句列表。
这里先把语句暂存在 `stmt_list` 里, 最后收束成一个 `CompoundStatementNode`。

4.3 stmt

位置:

- [src/parser.y:495](#)

怎么讲:

`stmt` 这一层把不同语句类型分发出去, 比如赋值、过程调用、`if`、`while`、`repeat-until`、`for`。
每个分支的动作代码都直接构造相应 AST 节点。

4.4 repeat_stmt

位置:

- [src/parser.y:534](#)

怎么讲:

`repeat ... until` 扩展也是在语法分析层先建树: 先收集循环体语句, 再在遇到 `UNTIL` 条件时构造 `RepeatUntilStatementNode`。

这段对“新功能不是只加 token”很有帮助。

4.5 ParserState

位置:

- [src/parser_state.h:9](#)
- [src/parser_state.cpp:11](#)

怎么讲:

这是 parser 的上下文状态。函数参数、语句列表、局部变量声明、数组信息和 record 信息都先在这里暂存, 再在合适的归约点落成 AST 节点。

4.6 `yyerror()` + `finish_syntax_recovery()`

位置:

- [src/parser.y:47](#)
- [src/parser.y:730](#)

怎么讲:

`yyerror()` 负责在第一次发现该错误位置时记录语法错误并进入 panic-mode。

`finish_syntax_recovery()` 负责清理 parser 的侧带状态, 并告诉错误处理模块当前这段恢复结束, 可以准备下一处独立错误。

5. 如果老师一个点一个点追问, 怎么答

5.1 语法分析和词法分析的接口是什么?

回答:

词法分析器通过 `yylex()` 向 Bison 提供 token 类型、语义值和位置信息, `yyparse()` 再根据文法进行归约。

5.2 为什么 `parser.y` 里还会有一些数组、record 的信息?

回答:

因为 parser 在建树时需要把这些结构信息一起挂到节点上, 比如数组维度、record 字段表, 所以会通过 `ParserState` 暂存这些侧带数据。但真正的合法性检查仍然放在语义分析阶段。

5.3 语法错误后为什么还能继续往后报?

回答:

因为我们在若干同步点实现了 panic-mode 恢复。也就是遇到错误后不是整份文件立刻停掉, 而是在分号、`END`、`UNTIL` 这类边界处恢复, 再继续分析后面的独立语法片段。

5.4 语法分析阶段会输出什么给后续?

回答:

输出是一棵 AST, 也就是内存中的节点结构, 不是文本。语义分析和代码生成都基于这棵树工作。

6. 老师如果让你“讲设计思路”, 你就这么收

我们这一层的设计思路是让 parser 专注于按文法建 AST, 把符号表和类型检查从 parser 里剥离出去。这样语法分析只关心结构是否成立, 同时配合 `ParserState` 管理解析期临时状态, 并通过有限的 panic-mode 恢复提高语法错误诊断的完整性。

7. 最短保底版：30 秒回答

如果他非常紧张，只背这段也够过关：

我主要参与的是语法分析与解析状态这一层。我们用 Bison 根据 Pascal-S 文法把 token 归约成 AST，输出是统一的抽象语法树，不是目标代码。解析过程中用 `ParserState` 暂存参数、语句列表、数组和 record 等侧带信息；语法错误时则通过 `error` 产生式和同步点做有限的 panic-mode 恢复。相关代码主要在 `parser.y` 和 `parser_state.*`。

8. 最容易说错的 4 个点

1. 不要把 parser 说成做了语义检查。
正确说法：parser 建树，语义分析做合法性检查。
2. 不要把 `ParserState` 说成符号表。
正确说法：它只是解析期上下文状态。
3. 不要把 panic-mode 恢复说成 `ErrorHandler` 做的。
正确说法：恢复在 parser 的 `error` 规则里，`ErrorHandler` 负责记录和输出。
4. 不要把 AST 说成文本输出。
正确说法：AST 是中间表示。

9. 周柄名只需要记住的文件定位

最推荐只记下面这些：

- [src/parser.y:21](#)
- [src/parser.y:185](#)
- [src/parser.y:461](#)
- [src/parser.y:480](#)
- [src/parser.y:534](#)
- [src/parser.y:730](#)
- [src/parser_state.h:9](#)

这 7 处足够覆盖他的大部分答辩问题。